

KĀDI: Knowledge & Ability Deployment Infrastructure

KASSI NZALASSE, Southern Methodist University, USA

HUNJAE LEE, Southern Methodist University, USA

COREY CLARK, Southern Methodist University, USA

As AI agents take on more of the scientific research lifecycle, the range of infrastructure they need grows with them. Integrating each new capability costs engineering time, fills the agent’s context with competing tool descriptions, and produces work with limited reusability across the lab. This paper presents KĀDI (Knowledge and Ability Deployment Infrastructure), an open-source, distributed infrastructure where capabilities published by one researcher become discoverable and invocable by collaborators and their agents through a single interface, with publisher-controlled visibility. We demonstrate KĀDI on three ongoing research projects, each in a different domain, using the same infrastructure instance for all three. Across all workflows, the agent’s integration surface remained constant while the capabilities behind it grew from a single storage backend to dynamically provisioned GPU compute to a psychometric experiment with 14,400 controlled trial responses from language models. Together, the three deployments show that work published to the infrastructure persists and accumulates across workflows, that distinct agents build on each other’s stored results without explicit coordination, and that the shared substrate expands the set of research questions a lab can feasibly pursue. Experiment descriptions and execution traces are included as supplementary materials. The KĀDI source code is publicly available.

CCS Concepts: • **Information systems** → **Collaborative and social computing systems and tools**; *Computing platforms*; • **Computing methodologies** → *Multi-agent systems*; *Cooperation and coordination*.

Additional Key Words and Phrases: multi-agent infrastructure, broker-mediated discovery, AI-enabled scientific research, multi-tenant capability sharing, cross-language composition, research infrastructure, provenance, open-source software

ACM Reference Format:

Kassi Nzalasse, Hunjae Lee, and Corey Clark. 2026. KĀDI: Knowledge & Ability Deployment Infrastructure. *ACM Trans. Manag. Inform. Syst.* 1, 1 (May 2026), 19 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Artificial intelligence is reshaping how scientific research is conducted. Autonomous agents have planned and carried out chemical syntheses in cloud laboratories [7], composed specialized chemistry tools for synthesis planning [8], and generated a research manuscript that passed peer review [30, 61]. Recent surveys trace a trajectory in which agents move beyond individual tasks to orchestrate entire research cycles, from generating hypotheses through designing and running experiments to refining results [56]. As agents take on more of the research lifecycle, however, they require access to increasingly heterogeneous infrastructure: databases for persistent storage, compute providers for GPU-intensive training, model services for inference, and domain-specific instruments [24]. Each new infrastructure

Authors’ Contact Information: [Kassi Nzalasse](mailto:knzalasse@smu.edu), knzalasse@smu.edu, Southern Methodist University, Dallas, Texas, USA; [Hunjae Lee](mailto:hunjael@smu.edu), hunjael@smu.edu, Southern Methodist University, Dallas, Texas, USA; [Corey Clark](mailto:coreyc@smu.edu), coreyc@smu.edu, Southern Methodist University, Dallas, Texas, USA.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

Manuscript submitted to ACM

capability an agent needs typically requires its own SDK, its own authentication, and its own allocation of the agent's limited context window [27, 36].

Scientific research is inherently collaborative [60], and this collaboration increasingly involves not just other researchers but autonomous agents from different providers operating across shared infrastructure [17, 56]. When a second researcher's agent needs to build on the first researcher's work, conventions that were implicit become opaque, tool configurations that were local become inaccessible, and the cost of onboarding a new participant grows with every capability already in place. Standardization efforts such as the Model Context Protocol [15, 21] have made meaningful progress toward uniform tool interfaces, and questions of multi-tenant capability sharing, distributed deployment, and horizontal scaling are active areas of development in the protocol ecosystem [1]. Chan et al. [10] formalize the broader category of *agent infrastructure*, identifying attribution, interaction shaping, and harm remediation as functions such infrastructure must serve. A parallel concern is that per-agent integration may narrow what gets explored: when each researcher's tools are siloed, agents default to whatever capabilities their individual integration provides, and the collective breadth of inquiry contracts [18, 32]. In a research lab where each agent is integrated against the capabilities its researcher needs, the unit of contribution becomes the workflow that researcher built, and workflows do not compose. A capability published once and reachable by every participant on the network is a different kind of contribution: it accumulates.

This paper presents KĀDI (Knowledge and Ability Deployment Infrastructure), an open-source, distributed infrastructure that makes capabilities published by one researcher discoverable, composable, and invocable by collaborators and their agents through a single interface, with publisher-controlled visibility. KĀDI does not replace the systems researchers already use; it sits beneath them. Agent frameworks [20, 42, 59], biomedical research ecosystems such as ToolUniverse [16], federated research platforms such as Academy [24], and decentralized compute marketplaces [2, 38] can all register as capabilities on the broker. Once registered, they become reachable by every connected participant through the same invocation. KĀDI does not prescribe how agents reason, plan, or model; it provides the substrate on which any agent or framework composes capabilities published by anyone on the network.

We demonstrate KĀDI on three ongoing research projects in different domains, using the same infrastructure instance for all three. The infrastructure exposed a constant integration surface (one broker connection, one invocation pattern, one identity) while the capabilities behind it spanned storage, dynamically provisioned compute, and model inference across multiple providers. One collaborator's research direction exists because the infrastructure made it tractable; it now stands co-equal in a grant proposal rather than listed as future work.

Our contributions are as follows:

- (1) A multi-tenant infrastructure model in which capabilities published by one researcher become discoverable, composable, and invocable by every agent on the network, while existing agent frameworks can connect to the broker as participants rather than being replaced.
- (2) A distributed transparency model in which abilities running locally, in the cloud, or on third-party infrastructure are reachable through one consistent interface, without the agent needing to know where they run.
- (3) A cross-language ability system in which abilities written in different programming languages are loaded and invoked through a single, language-agnostic interface.
- (4) A governance model in which every broker-mediated invocation is cryptographically attributable to a persistent agent identity, capabilities are published under publisher-controlled visibility scopes, and the entire stack can be self-hosted and operated air-gapped without dependency on any third-party service.

Section 4 demonstrates these contributions across three deployments on a single shared broker: capability accumulation across workflows (Case 1), cross-agent continuity across three model providers (Case 2), and a research arm enabled because the collaborator could compose capabilities published by other lab members (Case 3). Section 4.4 reports the discovery surface across all five runs.

2 Related Work

Agent frameworks assume co-location. Advances from the transformer architecture [53] through in-context learning at scale [9], instruction following [39], and chain-of-thought reasoning [55] have enabled a generation of LLM-based agent frameworks. Wang et al. [54] distinguish classical multi-agent systems [58] from this new class, in which agents leverage the reasoning capabilities of large foundation models [37]. The resulting agent frameworks [17, 20, 42, 59] have advanced planning, tool use [43, 48, 62], memory, and multi-agent coordination considerably. These systems are designed, however, for a single researcher orchestrating agents within one context: capabilities built for one project do not become available to agents in another, and the integration a researcher performs for their workflow is not reusable by colleagues. Wang et al. identify the deployment of multi-agent systems across heterogeneous environments as an open research direction, noting that architectures for distributing agents and their capabilities across devices and cloud infrastructure remain underexplored.

Communication protocols solve part of the problem. Several protocols have emerged to standardize how agents access tools. The Model Context Protocol (MCP) provides a JSON-RPC client-server interface for tool invocation [19, 21]. The Agent-to-Agent Protocol (A2A) adds peer-to-peer task delegation via capability-based Agent Cards, and the Agent Network Protocol (ANP) extends to decentralized discovery using W3C identifiers [15]. Together, these protocols address complementary tiers of agent communication [1]. MCP’s architecture operates on one-to-one connections between a client and a server: each agent must establish and maintain a direct connection to every server whose tools it needs. This design serves single-user tool integration well but does not inherently provide tenant isolation, horizontal scaling through a shared coordination point, or a distributed identity model for authentication across organizational boundaries. An empirical study of 1,899 open-source MCP servers found that 7.2% contain security vulnerabilities and 5.5% are susceptible to tool poisoning [19], underscoring that governance and trust mechanisms for shared tool ecosystems are still developing.

Tool surfaces grow faster than agents can navigate them. As tool catalogs grow, agent performance degrades. Tool selection accuracy drops sharply as catalogs grow into the thousands [36, 44], large tool responses can overflow the context entirely [27], and irrelevant tool information induces incorrect reasoning through what Roig [47] identifies as context pollution. Existing solutions (retrieval, graph-structured discovery, selective inclusion) assume a single user curating a known tool set [27, 36, 44]. The problem compounds when researchers want to share capabilities: every tool a new contributor publishes becomes visible to every agent on the shared infrastructure, and the literature documents significant performance degradation at catalog sizes well within reach of a small research group [29, 36]. Per-agent retrieval helps individual agents; the architectural question of how to keep a growing, multi-contributor tool ecosystem manageable without flooding any single agent’s context remains open.

Distributed compute solves a different layer. Distributing computation across clusters and cloud infrastructure is well-studied. Ray [35] provides task-parallel and actor-based computation, Pegasus [12] maps workflow descriptions onto distributed resources with provenance capture, and Parsl [4] augments Python with parallel constructs that scale

to supercomputer-class infrastructure. These systems schedule tasks whose definitions are known in advance. Making heterogeneous capabilities discoverable and invocable by agents that do not know where those capabilities run, or what capabilities exist, is a separate problem.

AI-for-science platforms occupy specific layers. Domain-specific agentic systems for chemistry [7, 8] and manuscript generation [30, 61] build purpose-built infrastructure per workflow, and experiment management platforms track results for human researchers but do not provide infrastructure through which agents discover and compose capabilities at runtime.

Academy [24] provides middleware for deploying stateful agents across federated HPC infrastructure, integrating with executors like Parsl [4] and Globus Compute. Academy is Python-native by design and shares KADI’s diagnosis that existing frameworks are inadequate for scientific infrastructure. Academy and KADI occupy different layers: an Academy deployment could register as a capability on a KADI broker, becoming reachable by every agent in a participating lab through the same interface used for every other capability. Gao et al. [16] present ToolUniverse, an ecosystem of over 600 tools for building AI scientists, with an MCP-based interaction protocol evaluated on a drug discovery case study. ToolUniverse standardizes tool access within its ecosystem. A ToolUniverse MCP server could similarly register on a KADI broker, making its tools reachable by every agent on the network alongside capabilities such as decentralized compute.

In the information systems literature, Dumas et al. [13] argue that process-aware AI systems require shared infrastructure for coordination, monitoring, and governance rather than ad-hoc integration. Zoubek et al. [65] propose a service architecture for synchronizing research data management across institutional stakeholders, and Chau and Xu [11] survey the IS research agenda for LLM-based systems including agentic workflows.

KADI’s position. In Wang et al.’s taxonomy [54], KADI sits below both classical and LFM-based multi-agent systems: it is the infrastructure layer that any agent or framework connects to. The systems reviewed above, agent frameworks, communication protocols, compute schedulers, AI-for-science platforms, each occupy a specific layer. KADI is the substrate through which any of them, and capabilities researchers have not yet built, become composable through one interface for every participant on the network.

3 KADI Infrastructure

KADI (Knowledge and Ability Deployment Infrastructure) is an open-source, distributed infrastructure for building, deploying, and orchestrating autonomous AI agents across machines and organizational boundaries. KADI handles the research infrastructure capabilities (storage, compute, model inference) that agents need across those boundaries. The agent’s own runtime (shell, file operations, code edits) is out of scope. The two compose without overlap, and the boundary between them defines the engineering scope of the infrastructure.

The design aligns with the FAIR principles [57] (abilities are findable through broker-mediated discovery, accessible via uniform protocol, interoperable across languages, and reusable as versioned packages) and rests on distribution by default, location transparency [52], outbound-only connections, and infrastructure separation from intelligence. Each is elaborated in the subsections that follow.

KADI organizes functionality into three layers. At the base, **tools** are deterministic functions with no embedded intelligence: an API call, a database query, a file operation. Tools are defined by a name, a JSON Schema describing their input, and a handler function. **Abilities** wrap tools with intelligence and package them for reuse. An ability may use a language model to select among tools, interpret results, handle errors adaptively, or chain operations. Each ability is

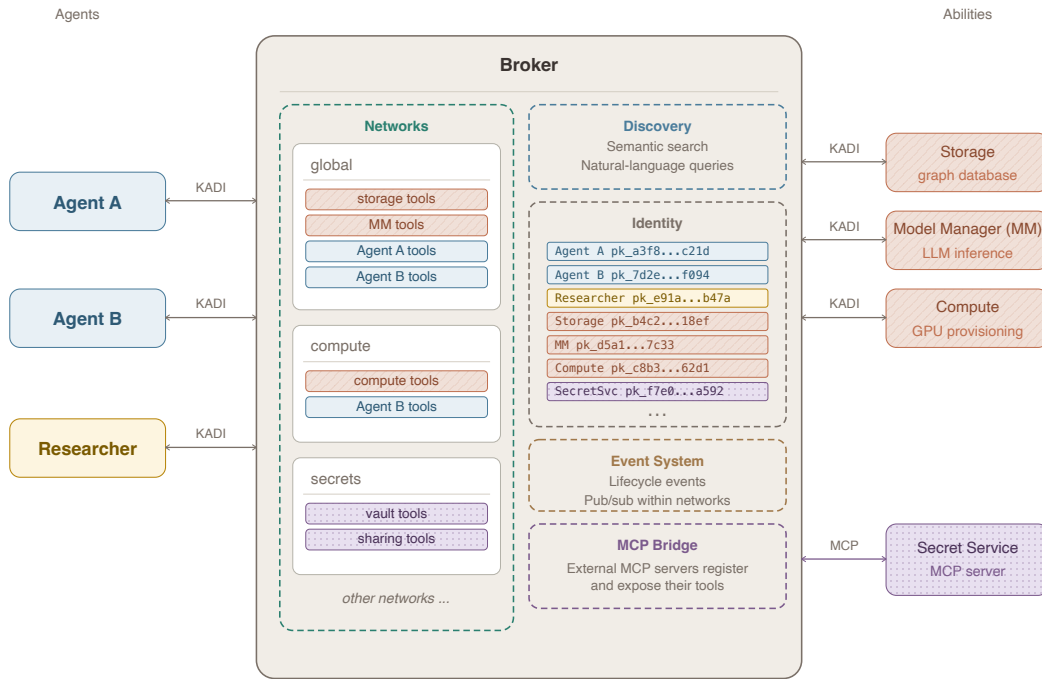


Fig. 1. KADI architecture. Agents (left) and abilities (right) connect to the broker via bidirectional, client-initiated connections using the KADI protocol (JSON-RPC over WebSocket) or MCP. The broker organizes tools into networks that scope visibility: a tool registered on a network is discoverable only by participants who have joined that network. Agent A's and Agent B's tools both appear in the global network; Agent B's tools also appear in compute because it participates in both. Every participant receives a persistent Ed25519 identity (shown in the Identity subsystem); every broker-mediated invocation is cryptographically attributable. The MCP Bridge allows external MCP servers, such as the Secret Service, to register and expose their tools to participants on the broker's networks. Tool boxes inside networks are color-matched to the participant that published them (diagonal pattern for native abilities, dotted pattern for MCP services, plain for agents). The Discovery subsystem supports semantic and natural-language queries over registered tools. The Event System provides lifecycle notifications and pub/sub messaging within networks.

characterized by a package manifest (`agent.json`) specifying its name, version, dependencies, and the tools it exposes. **Agents** are abilities running as standalone services connected to the broader ecosystem. The distinction is contextual: the same code operates as a library when imported into another project and as an independent service when deployed to a broker. This duality enables a natural development workflow: build and test locally, then deploy as a distributed agent without modifying application code.

3.1 Broker, Event System, and Networks

Brokers are the coordination layer of KADI deployments. When an ability connects to a broker, it registers the tools it provides. Other agents connected to the same broker can then discover and invoke those tools without knowing which agent provides them or where that agent runs. The broker routes requests to an appropriate provider and returns results transparently, though agents can specify a preferred provider when needed. Brokers speak both the native KADI protocol (JSON-RPC 2.0 [23] over WebSocket [31]) and the Model Context Protocol. This dual-protocol support operates in both directions: any MCP-compatible client (such as Claude Desktop) can connect to the broker and invoke

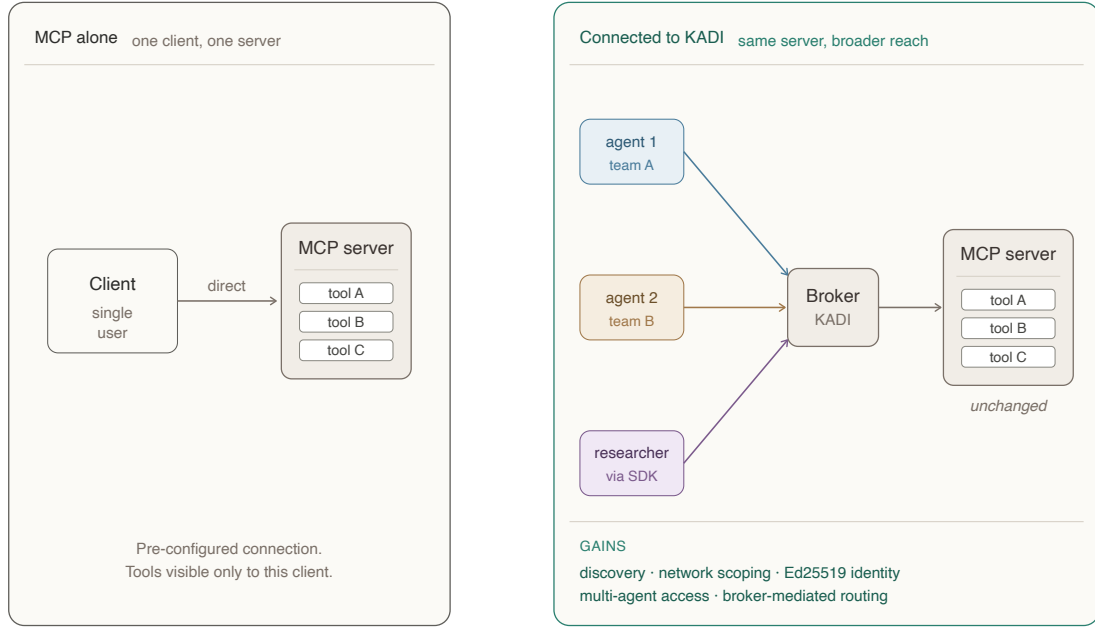


Fig. 2. An MCP server before and after connecting to the KADI broker. Left: standard MCP topology where a single client reaches the server through a direct, pre-configured connection. Right: the same server, unchanged, connected through the broker. Multiple agents from different teams and a human researcher using the SDK all reach the server's tools through the broker, which adds runtime discovery (agents find tools by querying, not by pre-configuration), network scoping (the operator controls which participants see which tools), and Ed25519 identity (every invocation is cryptographically attributable). The MCP server required no modification. KADI does not replace MCP; it extends the reach of existing MCP servers by connecting them to a shared, governed network where both agents and human researchers are participants.

tools provided by KADI agents, and the broker can connect as a client to existing MCP servers, listing their tools and re-exposing them to KADI agents with the same discovery, network scoping, and identity that native abilities receive. In the demonstration described in Section 4, an Akash Network MCP server was integrated through this mechanism, its tools becoming discoverable alongside native KADI abilities without modification to any agent.

Each broker organizes its namespace into **networks**, which serve as access control boundaries. When agents connect, they specify which networks to join, and they can only discover and invoke tools registered on those networks. This mechanism provides governance over agent capabilities without requiring complex role-based policies: the researcher deploying the agent decides which networks it joins, and the broker enforces visibility accordingly. By default, the agent does not need to know which provider handles a request, where it runs, or what language it is written in.

The broker publishes lifecycle events when agents connect, disconnect, or register new tools. Authorized observers can subscribe to this stream to reconstruct the current state of participants, tools, and networks on the broker, providing administrative oversight without separate monitoring infrastructure. Agents can also publish and subscribe to events within the networks they have joined, enabling coordination between participants as a built-in property of the

architecture. The supplementary materials include topology visualizations captured from the broker’s event stream, showing real-time administrative visibility into connected participants, networks, and registered tools.

3.2 SDK and Cross-Language Composition

KADI exposes one mechanism for loading abilities, with three composition modes that differ in deployment but present an identical invocation interface. *Native* loading imports the ability directly into the caller’s process for maximum performance. *Stdio* loading runs the ability as a subprocess communicating over standard input/output, following the precedent of the Language Server Protocol [33]. *Broker* loading invokes the ability through a broker for fully distributed operation, in the lineage of polyglot microservice architectures [22]. An ability written in Python can be called from a TypeScript agent; an ability running on a remote machine can be called the same way as one imported in-process. The choice of mode is a deployment decision, not an application-code change. Human researchers use the same SDK to invoke broker capabilities programmatically, accessing the same tools agents use. Table 1 summarizes the three modes and their tradeoffs. All three deployments in this paper used broker loading, as the workflows required distributed infrastructure.

Table 1. Connection mode tradeoffs. Each mode presents the same invocation interface. The choice depends on deployment requirements, not application code.

	Native	Stdio	Broker
Cross-language	No	Yes	Yes
Cross-machine	No	No	Yes
Multi-agent	No	No	Yes

3.3 Distributed Transparency and Sovereign Deployment

A core design choice is that only brokers require public network addresses. Agents and abilities connect outbound to brokers; brokers never initiate connections to agents. Agents can operate behind corporate firewalls, on residential networks, or inside restrictive cloud security groups without requiring open ports or VPN configuration. The practical consequence is that distributed agent systems can be assembled by small teams without enterprise networking infrastructure.

KADI can be deployed on user-controlled infrastructure (Figure 3) or on third-party cloud providers. The supplementary materials include a screenshot of an independent broker instance deployed on DigitalOcean by another lab member. KADI also integrates natively with Akash Network [2, 38], a decentralized compute marketplace, enabling deployment where no cloud provider account is required and no single entity controls access to the infrastructure. An agent can connect to multiple independent brokers simultaneously (Figure 3): a researcher may run a local broker under their own control while also participating in the lab’s shared broker, choosing which capabilities to develop locally and which to publish to shared infrastructure.

3.4 Provenance as Architectural Byproduct

Every tool invocation that crosses the broker carries a persistent Ed25519 [6] cryptographic identity. Each agent’s identity is derived deterministically from its public key and remains stable across sessions, so invocations from the same agent are attributable even if the agent reconnects. The broker tracks each invocation with a correlation identifier, the

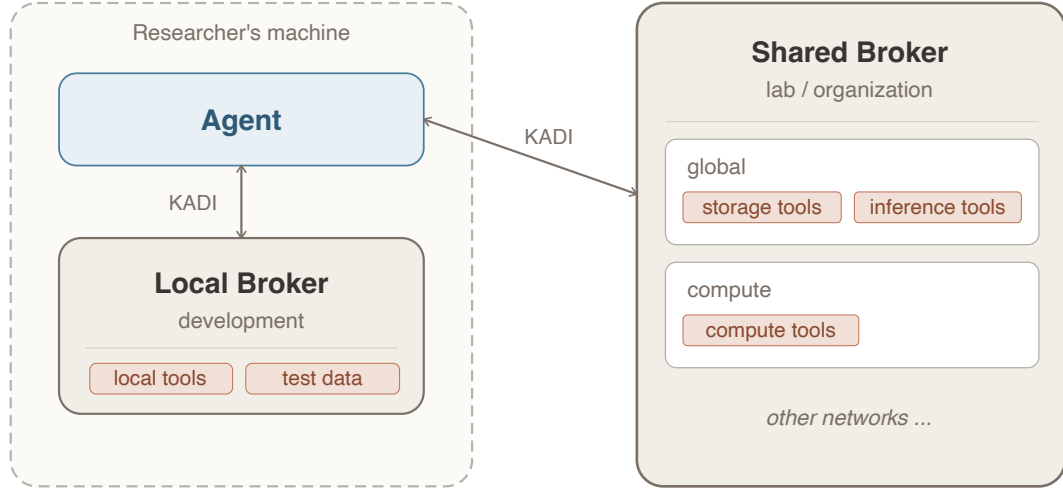


Fig. 3. An agent connecting to two independent brokers simultaneously. The researcher runs a local broker on their own machine for development, while the same agent also participates in the lab’s shared broker. The researcher controls which capabilities remain local and which are published to shared infrastructure. Both connections use the same JSON-RPC-based KADI protocol.

caller’s identity, the tool name and parameters, timestamps, duration, and completion status. When the broker forwards a request to a provider, it injects the caller’s cryptographic identity into the request metadata, making the full invocation chain attributable end-to-end. Provenance records are a natural byproduct of the broker-mediated architecture, not a separate instrumentation layer that must be configured and maintained.

The provenance literature has established formal models for representing causal chains in workflows, from the Open Provenance Model [34] to the W3C PROV standard [5]. Established systems such as Pegasus [12] capture provenance through per-task monitoring instrumentation, and recent work extends provenance to the agent loop itself [49] and to interactive analysis of runtime provenance data [50]. KADI captures provenance at the broker boundary automatically, without requiring agents to adopt a specific provenance framework.

This provenance covers broker-mediated invocations. Local operations (shell commands, file edits, code changes) are recorded by the agent harness in its own execution trace but do not cross the broker. Together, the two layers provide end-to-end provenance: the broker records what crossed the infrastructure boundary, and the harness records what happened locally.

4 Deployments

We demonstrate KADI on three ongoing research projects, each in a different domain with different infrastructure requirements, using the same KADI instance for all three: the same broker, the same shared storage infrastructure, the same agent harness, and the same KADI extension. In each case the agent received an experiment description file and a broker address. It was not prescribed which tools to use; it discovered and leveraged capabilities through the infrastructure as its task required.

All agents ran inside the pi-mono agent harness [63], an open-source coding agent providing built-in file, shell, and editing operations. We wrote two extensions for the harness: an SSH extension for remote machine access, and the KADI extension shown in Figure 4, providing one tool (`kadi_call`) through which the agent accessed every capability on the network. Abilities connect to the broker over the KADI protocol or via MCP; the language they are written in is transparent to the agents that invoke them. The experiment descriptions given to each agent and the complete execution traces are released as supplementary materials.

4.1 Database Migration

A collaborator had built a knowledge graph of 2,281 entities and 3,328 relationships as part of a research project, stored in a Neo4j instance that had reached its free-tier storage limits. The researcher needed this graph migrated into the lab’s shared ArcadeDB instance, preserving all vertices, edges, and their properties. The agent received the source data as a JSON export, an experiment description file, and a broker address. The task required storage abilities only: schema creation, data import, and query validation.

The agent’s first actions were discovery calls. It issued 10 queries to the broker before executing its first write operation, requesting available tools, tool documentation, and guidance on SQL syntax from the database’s built-in assistant. It then designed a schema, created the target database, and began the migration. The full process ran end-to-end in approximately 22 minutes, from the agent’s first discovery call to its final verification query. The agent migrated 2,281 vertices and 3,328 edges, matching the source dataset exactly. It encountered 12 errors during execution and recovered from all of them autonomously, including a missing-toolchain fallback to Node.js native HTTPS and a self-diagnosed edge-migration gap (83 of 3,328 edges initially) that it repaired by replaying the missing relationships. The target database server is shared infrastructure used by other agents and services in the lab; the agent created its own database and confined its writes there, leaving all other databases untouched. Every operation, from schema creation through the repair pass, used the same broker invocation the agent would use for any other capability on the network.

Data migration is a recurring research chore. Researchers either do it manually each time or write bespoke scripts that rarely outlive the project they were written for. Through KADI, the agent completed the migration through storage abilities that were already on the broker before the migration began and remain on the broker after it finished. The next researcher whose agent needs to create a database, import data, or run queries will discover and use the same abilities without re-engineering them. The migration happened once. The capabilities it used persist for every subsequent workflow, every subsequent agent, and every subsequent researcher on the network. A workflow built once helps once. A capability published once accumulates.

4.2 Training with Dynamic Compute

A collaborator was training neural network models to predict physical system dynamics from simulation data. The codebase supported six model architectures and four simulated environments, and running experiments required GPU hardware not available locally. Three agents, GPT-5.4 (OpenAI), Kimi-K2.6-thinking (Moonshot), and Claude Sonnet 4.6 (Anthropic), were deployed one after another to demonstrate agents building on prior work discovered through the broker. Each received the same experiment description file and the same broker address. The task required storage and dynamically provisioned compute.

The three agents ran independently with no shared runtime. Each began with a discovery phase: the first agent issued 8 consecutive discovery calls covering compute provisioning, database operations, and SSH access; the second and third issued 10 and 4 respectively, each time querying the broker for prior work left by earlier agents. The decentralized

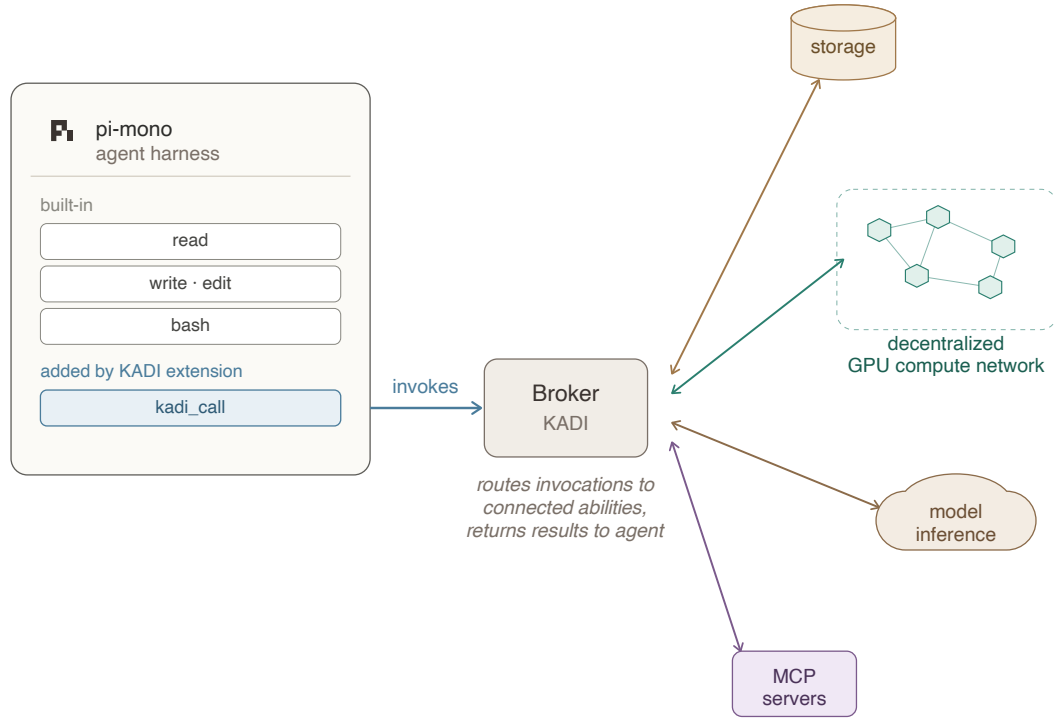


Fig. 4. Agent harness architecture used in all three deployments. The agent runs inside pi-mono, an open-source coding harness providing built-in tools for file operations, editing, and shell access. A single KADI extension adds one tool (`kadi_call`) through which the agent requests any capability on the network; the broker routes each request to the appropriate ability and returns the result. Through this one tool, the agent accesses storage abilities, the entire Akash decentralized compute network for on-demand GPU provisioning, model inference across cloud and self-hosted providers, and any MCP server connected to the broker. The left side of the diagram is the agent harness; the KADI extension (highlighted) is the only component we wrote. The right side is the capability surface the agent gains through that single extension. The same capabilities are also accessible programmatically through the KADI SDK, enabling human researchers to invoke the same tools from their own code.

GPU marketplace was reachable by the same `kadi_call` invocation used to read the database. From the agent's side, provisioning a GPU container was indistinguishable from any other capability on the broker.

The first agent ran 10 experiments across three environments, evaluated its results after each run, and recorded both findings and open questions in the shared database. The second agent's first broker calls were queries reading the first agent's stored data: it retrieved all prior session metadata, all ten experiment records, and the five most recent

reflections. It then identified that observation noise in the training data was confounding comparisons across model architectures, generated clean datasets with zero observation noise, and ran 7 targeted experiments, extending to a fourth environment (three-body gravitational dynamics) that the first agent had not attempted. The third agent retrieved all 17 prior experiments and reflections from both predecessors. Its reasoning trace shows explicit gap analysis: it noted that LSTM and velocity model architectures had never been tested across any environment and that no experiments had been run at extended training durations. It systematically filled both gaps, running 22 experiments covering all six architectures across all four environments at both 25 and 50 epochs. It also recognized a recurring code bug that prior agents had encountered and already patched. In total, the three agents ran 39 experiments over approximately 153 minutes of combined execution (Figure 5).

Each agent was instructed to check the shared database for prior work before starting, but what to do with what it found was not specified. Because all experiment records and reflections were stored in a shared database accessible through the same broker interface the agent used for every other operation, building on a predecessor’s work required no special tooling, no data export, and no manual handoff. What occurred across these three agents resembles what Kitcher [26] calls the division of cognitive labor, enabled by broker-mediated discovery: prior work reachable through the same interface used for every other capability. Agent 2 identified a methodological confound in Agent 1’s results that Agent 1 had not detected. Agent 3 identified two model architectures that neither predecessor had tested. Each agent, given the same task with no prescribed workflow and no shared runtime, made different research choices and produced complementary rather than redundant contributions. Hao et al. [18] document that AI tools narrow collective scientific focus by concentrating work in data-rich areas. The mechanism these three agents demonstrated runs in the opposite direction: shared infrastructure preserves the methodological diversity its participants bring, because the cost of building on prior work is reduced to a database query.

4.3 Automated Psychometric Experiment

Education researchers collaborating with our lab had developed a psychometric instrument designed to measure the natural number bias in children [45], and wanted to investigate whether the same bias appears in language models. The researcher needed this instrument administered to six language models: four closed-source models from two providers (OpenAI and Anthropic) and two open-source models available via ollama (Qwen 3.5 27B and Gemma 4 26B), under two experimental conditions (a baseline with no priming and a condition where the model was given a persona that mimics the reasoning bias the instrument measures), with thirty trials per item per model per condition to ensure statistical reliability.

The agent received the same inputs: an experiment description file and a broker address. The task required all three ability families simultaneously: compute (GPU containers for self-hosted model inference, provisioned on demand from the same decentralized marketplace as the previous deployment), model inference (a gateway routing requests to multiple cloud providers, handling provider-specific authentication and rate limiting transparently), and storage (the same shared graph database used in both previous deployments). The agent issued 9 discovery calls and used 12 distinct broker-exposed tools. It collected 14,400 trial responses in approximately 105 minutes. Only 2 of 14,400 responses were unparseable (99.99% parse rate). The agent recovered from 14 execution errors, including SSH disconnections and background process failures on the ephemeral compute machine. It managed an A100 GPU session over approximately two hours, handling 8 SSH reconnections and adapting to provider instability.

This research would not have been undertaken without KADI. The collaborators had neither the infrastructure nor the engineering bandwidth to orchestrate cross-provider inference, on-demand GPU provisioning on a decentralized

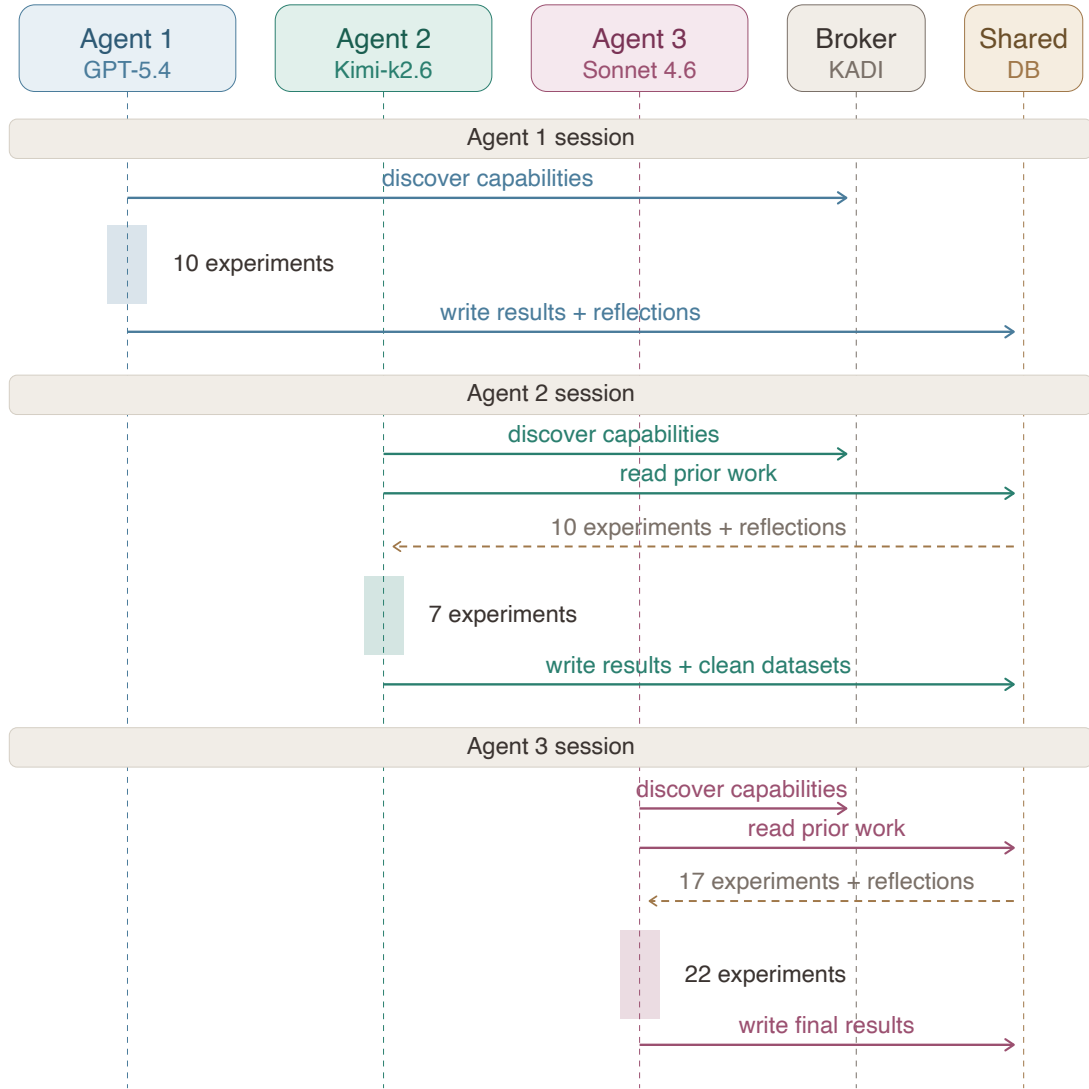


Fig. 5. Cross-agent continuity in the training case. Three agents from different model providers (GPT-5.4 from OpenAI, Kimi-K2.6 from Moonshot, Claude Sonnet 4.6 from Anthropic) ran independently on the same task, each in its own session with no shared runtime. Each began with a discovery phase (8, 10, and 4 broker queries respectively). Agent 1 found no prior work and established the initial experiment set. Agents 2 and 3 each read the full record of prior experiments and reflections from the shared database and decided independently what to investigate next: Agent 2 identified that Agent 1’s results were confounded by observation noise and generated clean datasets; Agent 3 identified two untested architectures and ran a systematic sweep covering all six architectures across all four environments. Every interaction between an agent and the shared database was mediated by the broker, which routed each request to the database ability and returned the result. The agents also provisioned GPU compute through the same broker (omitted from diagram for clarity). No agent accessed the database directly; no agent communicated with another agent.

marketplace, and 14,400 counterbalanced trials. The compute, model inference, and storage capabilities the agent used in this deployment were not built by the collaborators; they were published by other lab members and available on the broker. The collaborators’ research question became tractable because the broker absorbed the operational complexity, leaving the agent free to enforce the protocol’s methodological constraints. The result was not just an enabled experiment but a methodologically rigorous one: 14,400 counterbalanced trials and a 99.99% parse rate. In the resulting grant proposal, the LLM comparison arm now sits co-equal with the human study rather than appended as future work. KADI’s value here is not acceleration; it is that the broker absorbed the operational complexity that would otherwise have made this research arm intractable for a group without engineering staff.

Across the three deployments, two cross-cutting observations follow from the execution traces: how agents navigated the broker’s tool surface (Section 4.4) and how the same infrastructure served all three workflows without reconfiguration (Section 4.5).

4.4 Discovery and Composition

Section 2 identified the open question of how to keep a growing, multi-contributor tool ecosystem navigable without flooding any single agent’s context. At the time of the demonstration, the broker hosted 161 tools across 6 networks from 17 connected agents. Across all five runs, the agents discovered and used 18 distinct tools (ranging from 7 to 12 per run), as verified by the execution traces. The remaining tools were never surfaced: network scoping restricted visibility to joined networks, and the broker’s progressive discovery mechanism let agents ask natural-language questions (e.g., “what tools can help me provision GPU compute?”) and receive filtered, ranked results rather than a complete catalog. Table 2 summarizes the tool surface per run. The effective working surface of 7–12 tools per run, out of 161 available, was achieved without any per-agent retriever; the structural mechanism is the broker. Across all five runs, no agent specified a provider, a host, or a language for any tool it used; each invocation went through the same `kadi_call` interface regardless of where the ability ran or what language it was written in. The broker’s network scoping and progressive discovery kept each agent’s working surface small even as the catalog grew.

Table 2. Broker tool surface per run. The broker hosted 161 tools across 6 networks from 17 connected agents at the time of the demonstration. Each agent was given no tool list; it discovered capabilities through the broker’s discovery mechanism. Network scoping (column 2) restricted each agent’s visibility to joined networks. The rightmost two columns show the contrast: the number of distinct broker-exposed tools each agent actually used versus the 161 available. Across all five runs, agents used 18 distinct tools (11% of the catalog). Data verified from execution traces included in the supplementary materials.

Run	Networks joined	Broker tools used	Available on broker
Case 1: Migration	1	11	161
Case 2: Training (Ag. 1)	3	8	161
Case 2: Training (Ag. 2)	3	7	161
Case 2: Training (Ag. 3)	3	7	161
Case 3: Experiment	3	12	161

4.5 Multi-Tenant Reuse

All three deployments above ran on the same KADI instance, self-hosted on a lab server with no external dependencies beyond the Akash compute marketplace and the cloud model providers used in the third deployment. The storage abilities the agent used in the first deployment were the same abilities it used in the second and third. No reconfiguration

was required between deployments. The same ArcadeDB server hosted all three, each deployment writing to its own database. The experiment agents were not the only participants on the broker: abilities published by other lab members were running concurrently on the same instance, and the experiment agents operated alongside them without interference. The observations across all three deployments were identified through manual inspection of the complete execution traces. The supplementary materials include these traces for independent verification, along with topology visualizations from multiple independent broker deployments, each shaped by the needs of its participants.

5 Discussion

5.1 When Sharing Is the Default, Contributions Compound

Star and Ruhleder [51] observe that in networked infrastructure, each new participant increases the resources available to every other participant. The broker instantiates this: each new ability increases the capability surface for every participant with access, without per-user configuration.

Ethnographic studies of scientific cyberinfrastructure have shown that sharing in conventional distributed projects requires extensive human coordination across policies, standards, and tools [28]. Edwards et al. [14] identify the “gateway problem” as central to infrastructure evolution: isolated systems interoperate only when gateway technologies bridge them. In KADI, the broker serves this gateway function, and the human coordination cost is absorbed into the technical layer: JSON Schema on each tool serves as the standard, network scoping serves as the access policy, and broker-mediated discovery replaces per-user documentation. Sharing is the default, and governance is built into the sharing mechanism. A researcher publishing an ability decides, per tool, which broker networks that tool appears on; the researcher deploying an agent decides which networks that agent may join. The broker enforces both sides.

The database migration in Section 4.1 is one instance of this pattern: when a capability is published to the broker, the cost of sharing does not scale with the number of beneficiaries. When an existing ability on the network improves, every participant benefits from the improvement without reconfiguration. The unit of contribution shifts from the workflow a researcher builds to the capability they publish to shared infrastructure, because only the latter compounds. Because the broker does not distinguish between human and agent contributors, this compounding extends to both. A researcher using the KADI SDK can read an agent’s stored results and continue the work with the same tools.

5.2 Participant Diversity Preserved at the Infrastructure Layer

Messeri and Crockett [32] argue that the proliferation of AI tools in science risks creating “scientific monocultures” in which dominant methods crowd out alternative approaches. Empirical work has confirmed this concern: Hao et al. [18] analyze 41.3 million papers across natural science disciplines and find that AI adoption expands individual scientists’ output but contracts the collective breadth of topics studied. Anderson et al. [3] show at the mechanism level that different users of the same LLM produce more semantically similar ideas than users of alternative tools.

The training case (Section 4.2) offers suggestive evidence that shared infrastructure may counter this mechanism at small scale. Three agents from three different model providers, given the same task with no prescribed workflow and no shared runtime, made different research choices and produced complementary rather than redundant contributions. The broker delivers capabilities but exerts no opinion on which models, tools, or research directions agents pursue, separating capability delivery from scientific methodology.

We do not claim that infrastructure alone resolves the convergence pressure Hao et al. document at the scale of millions of papers. The finding is suggestive at small scale: three agents, three providers, one task. Where a per-agent

integration model homogenizes contributions to whatever each model defaults to, broker-mediated capability sharing makes prior work cheap to discover and extend, leaving each new participant free to contribute against the gaps they identify. Whether this property meaningfully affects diversity at the scale Messeri and Crockett describe remains an open empirical question.

5.3 Provenance and Reproducibility at the Architectural Boundary

The adoption of ML methods in scientific research has been accompanied by documented failures of validity, reproducibility, and generalizability [25]. Consensus reporting standards now identify detailed records of computational infrastructure, data provenance, and modeling decisions as essential for reproducible ML-based science [25, 41]. KADI's broker captures a subset of these records automatically: the identity of the agent invoking each capability, the parameters of each invocation, and the order of operations, without requiring authors to remember or curate them. Every tool invocation is attributable to a persistent cryptographic identity. Across the 923 tool calls in this paper's demonstration, the broker captured 39% under cryptographic attribution; the remaining 61% were local harness operations, recorded in the agent's execution trace but not crossing the broker. The complete execution traces, the shared experiment records, and the experiment descriptions together form an audit trail that allows any workflow to be inspected after the fact.

KADI fulfills Chan et al.'s [10] attribution function as a consequence of operational design rather than as an instrumented add-on: every broker-mediated invocation is attributable to a persistent cryptographic identity by virtue of how the infrastructure works. Following Pineau et al.'s terminology [41], KADI provides infrastructure that supports the conditions for reproducibility by recording what was done, with what tools, in what order. Whether reproducibility is achieved depends on factors outside the infrastructure's control: identical model versions, hardware-level numerical determinism, and stable infrastructure state. This governance extends to deployment boundaries, where a researcher may run a local broker under their sole control while participating in a shared broker (Figure 3), choosing which capabilities remain private and which are published to the network.

5.4 Scope and Trade-offs

The broker is a single point of coordination: agents depend on broker availability for distributed operation. This centralization is a design choice that enables central discovery, access control, and provenance without requiring agents to maintain peer-to-peer connections. The dependency can be mitigated through broker replication, and abilities loaded natively or via stdio operate without any broker at all. All three deployments ran on a single broker instance representing one research lab. KADI supports agents connecting to multiple independent brokers simultaneously, enabling cross-organizational federation, but this was not demonstrated in the present work.

The substrate the broker provides is open-ended. What registers as a capability is whatever runs and connects. The properties demonstrated here (compounding, participant diversity, enablement) are properties of broker-mediated capability sharing, not of these particular capabilities.

6 Future Work

Three extensions follow naturally from this work. First, the cross-agent continuity demonstrated in Section 4.2 used storage primitives alone; a cross-agent, cross-session memory layer that distills prior experience rather than querying raw records would let agents accumulate research intuition over time [40, 64], building on recent work in collaborative agent memory with provenance and access control [46]. Second, scaling to larger fleets of concurrent agents on related problems would exercise the broker's coordination capacity beyond what three agents demonstrated. Third, all three

deployments ran on a single broker representing one lab; cross-organizational federation, where agents connect to multiple independent brokers, would extend the capability ecosystem across institutional boundaries.

7 Conclusion

KĀDI is the infrastructure layer through which agent frameworks, tool catalogs, federated compute platforms, and MCP servers become composable through one interface for every agent and human researcher on the network. We demonstrated this on three ongoing research projects, where a constant integration surface absorbed growing capability complexity and agents from different model providers built on each other’s stored results. Experiment descriptions, execution traces, and broker topology visualizations are included as supplementary materials. The KĀDI source code is available.¹

Acknowledgments

Claude was used to help draft and refine prose across sections from author-provided outlines and analyses. All research design, analyses, claims, and citations are the authors’ own and were independently verified.

References

- [1] Apoorva Adimulam, Rajesh Gupta, and Sumit Kumar. 2026. The Orchestration of Multi-Agent Systems: Architectures, Protocols, and Enterprise Adoption. *ArXiv abs/2601.13671* (2026). <https://api.semanticscholar.org/CorpusID:284910104>
- [2] Akash Network. 2018. Akash Network: A Decentralized Cloud Computing Marketplace. Whitepaper. <https://akash.network/whitepapers/> Accessed: 2026-04-28.
- [3] Barrett R Anderson, Jash Hemant Shah, and Max Kreminski. 2024. Homogenization Effects of Large Language Models on Human Creative Ideation. In *Proceedings of the 16th Conference on Creativity & Cognition* (Chicago, IL, USA) (CC ’24). Association for Computing Machinery, New York, NY, USA, 413–425. doi:10.1145/3635636.3656204
- [4] Yadu Babuji, Anna Woodard, Zhuozhao Li, Daniel S. Katz, Ben Clifford, Rohan Kumar, Lukasz Lacinski, Ryan Chard, Justin M. Wozniak, Ian Foster, Michael Wilde, and Kyle Chard. 2019. Parsl: Pervasive Parallel Programming in Python. In *Proceedings of the 28th International Symposium on High-Performance Parallel and Distributed Computing* (Phoenix, AZ, USA) (HPDC ’19). Association for Computing Machinery, New York, NY, USA, 25–36. doi:10.1145/3307681.3325400
- [5] Khalid Belhajjame, Reza B’Far, James Cheney, Sam Coppens, Stephen Cresswell, Yolanda Gil, Paul Groth, Graham Klyne, Timothy Lebo, Jamie McCusker, Simon Miles, James D. Myers, Satya Sanket Sahoo, and Curt Tilmes. 2013. PROV-DM: The PROV Data Model. <https://api.semanticscholar.org/CorpusID:65235238>
- [6] Daniel J. Bernstein, Niels Duif, Tanja Lange, Peter Schwabe, and Bo-Yin Yang. 2012. High-speed high-security signatures. *J. Cryptogr. Eng.* 2, 2 (2012), 77–89. doi:10.1007/S13389-012-0027-1
- [7] Daniil A. Boiko, Robert MacKnight, Benjamin C. Kline, and Gabe Gomes. 2023. Autonomous chemical research with large language models. *Nature* 624 (2023), 570–578. doi:10.1038/s41586-023-06792-0
- [8] Andrés M. Bran, Sam Cox, Oliver Schilter, Carlo Baldassari, Andrew D. White, and Philippe Schwaller. 2024. Augmenting large language models with chemistry tools. *Nature Machine Intelligence* 6 (2024), 525–535. doi:10.1038/s42256-024-00832-8
- [9] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (Eds.), Vol. 33. Curran Associates, Inc., 1877–1901. https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfc4967418bfb8ac142f64a-Paper.pdf
- [10] Alan Chan, Kevin Wei, Sihao Huang, Nitarshan Rajkumar, Elija Perrier, Seth Lazar, Gillian K Hadfield, and Markus Anderljung. 2025. Infrastructure for AI Agents. *Transactions on Machine Learning Research* (2025). <https://openreview.net/forum?id=Ckh17xN2R2>
- [11] Michael Chau and Jennifer Xu. 2025. An IS Research Agenda on Large Language Models: Development, Applications, and Impacts on Business and Management. *ACM Trans. Manage. Inf. Syst.* 16, 1, Article 1 (Feb. 2025), 11 pages. doi:10.1145/3713032

¹<https://gitlab.com/humin-game-lab/kadi>

- [12] Ewa Deelman, Karan Vahi, Gideon Juve, Mats Rynge, Scott Callaghan, Philip J. Maechling, Rajiv Mayani, Weiwei Chen, Rafael Ferreira da Silva, Miron Livny, and Kent Wenger. 2015. Pegasus, a workflow management system for science automation. *Future Generation Computer Systems* 46 (2015), 17–35. doi:10.1016/j.future.2014.10.008
- [13] Marlon Dumas, Fabiana Fournier, Lior Limonad, Andrea Marrella, Marco Montali, Jana-Rebecca Rehse, Rafael Accorsi, Diego Calvanese, Giuseppe De Giacomo, Dirk Fahland, Avigdor Gal, Marcello La Rosa, Hagen Völzer, and Ingo Weber. 2023. AI-augmented Business Process Management Systems: A Research Manifesto. *ACM Trans. Manage. Inf. Syst.* 14, 1, Article 11 (Jan. 2023), 19 pages. doi:10.1145/3576047
- [14] Paul N. Edwards, Geoffrey C. Bowker, Steven J. Jackson, and Robin Williams. 2009. Introduction: An Agenda for Infrastructure Studies. *J. Assoc. Inf. Syst.* 10 (2009), 6. <https://api.semanticscholar.org/CorpusID:263754817>
- [15] Abul Ehtesham, Aditi Singh, Gaurav Kumar Gupta, and Saket Kumar. 2025. A survey of agent interoperability protocols: Model Context Protocol (MCP), Agent Communication Protocol (ACP), Agent-to-Agent Protocol (A2A), and Agent Network Protocol (ANP). *ArXiv abs/2505.02279* (2025). <https://api.semanticscholar.org/CorpusID:278327419>
- [16] Shanghua Gao, Richard Zhu, Pengwei Sui, Zhenglun Kong, Sufian Aldogom, Yepeng Huang, Ayush Noori, Reza Shamji, Krishna Parvataneni, Theodoros Tsiligkaridis, and Marinka Zitnik. 2025. Democratizing AI scientists using ToolUniverse. (2025). arXiv:2509.23426 [cs.AI] <https://arxiv.org/abs/2509.23426>
- [17] Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, N. Chawla, Olaf Wiest, and Xiangliang Zhang. 2024. Large Language Model based Multi-Agents: A Survey of Progress and Challenges. In *International Joint Conference on Artificial Intelligence*. <https://api.semanticscholar.org/CorpusID:267412980>
- [18] Qianyu Hao, Fengli Xu, Yong Li, and James Evans. 2026. Artificial intelligence tools expand scientists’ impact but contract science’s focus. *Nature* 649, 8099 (2026), 1237–1243. doi:10.1038/s41586-025-09922-y
- [19] Mohammed Mehedi Hasan, Hao Li, Emad Fallahzadeh, Gopi Krishnan Rajbahadur, Bram Adams, and Ahmed E. Hassan. 2025. Model Context Protocol (MCP) at First Glance: Studying the Security and Maintainability of MCP Servers. *ArXiv abs/2506.13538* (2025). <https://api.semanticscholar.org/CorpusID:279403291>
- [20] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiwu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zi Hen Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2024. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. In *International Conference on Learning Representations*. <https://api.semanticscholar.org/CorpusID:265301950>
- [21] Xinyi Hou, Yanjie Zhao, Shenao Wang, and Haoyu Wang. 2025. Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions. arXiv:2503.23278 [cs.CR] <https://arxiv.org/abs/2503.23278>
- [22] Pooyan Jamshidi, Claus Pahl, Nabor C. Mendonça, James Lewis, and Stefan Tilkov. 2018. Microservices: The Journey So Far and Challenges Ahead. *IEEE Software* 35, 3 (2018), 24–35. doi:10.1109/MS.2018.2141039
- [23] JSON-RPC Working Group. 2013. JSON-RPC 2.0 Specification. <https://www.jsonrpc.org/specification>. Accessed: 2026-04-28.
- [24] Alok Kamatar, J. Gregory Pauloski, Yadu Babuji, Ryan Chard, Mansi Sakarvadia, Daniel Babnigg, Kyle Chard, and Ian Foster. 2025. Empowering Scientific Workflows with Federated Agents. *ArXiv abs/2505.05428* (2025). <https://api.semanticscholar.org/CorpusID:278394651>
- [25] Sayash Kapoor, Emily M. Cantrell, Kenny Peng, Thanh Hien Pham, Christopher A. Bail, Odd Erik Gundersen, Jake M. Hofman, Jessica Hullman, Michael A. Lones, Momin M. Malik, Priyanka Nanayakkara, Russell A. Poldrack, Inioluwa Deborah Raji, Michael Roberts, Matthew J. Salganik, Marta Serra-Garcia, Brandon M. Stewart, Gilles Vandewiele, and Arvind Narayanan. 2024. REFORMS: Consensus-based Recommendations for Machine-learning-based Science. *Science Advances* 10, 18 (2024), eadk3452. arXiv:https://www.science.org/doi/pdf/10.1126/sciadv.adk3452 doi:10.1126/sciadv.adk3452
- [26] Philip Kitcher. 1990. The division of cognitive labor. *The journal of philosophy* 87, 1 (1990), 5–22.
- [27] Anton Bulle Labate, Valesca Moura de Sousa, Sandro Rama Fiorini, Leonardo Guerreiro Azevedo, Raphael Melo Thiago, and Viviane Torres da Silva. 2025. Solving Context Window Overflow in AI Agents. arXiv:2511.22729 [cs.AI] <https://arxiv.org/abs/2511.22729>
- [28] Charlotte P. Lee, Paul Dourish, and Gloria Mark. 2006. The human infrastructure of cyberinfrastructure. In *Proceedings of the 2006 20th Anniversary Conference on Computer Supported Cooperative Work (Banff, Alberta, Canada) (CSCW '06)*. Association for Computing Machinery, New York, NY, USA, 483–492. doi:10.1145/1180875.1180950
- [29] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranajpe, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2023. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics* 12 (2023), 157–173. <https://api.semanticscholar.org/CorpusID:259360665>
- [30] Chris Lu, Cong Lu, Robert Tjarko Lange, Yutaro Yamada, Shengran Hu, Jakob Foerster, David Ha, and Jeff Clune. 2026. Towards End-to-End Automation of AI Research. *Nature* 651, 8107 (2026), 914–919. doi:10.1038/s41586-026-10265-5
- [31] Alexey Melnikov and Ian Fette. 2011. The WebSocket Protocol. RFC 6455. doi:10.17487/RFC6455
- [32] Lisa Messeri and M. J. Crockett. 2024. Artificial intelligence and illusions of understanding in scientific research. *Nature* 627 (2024), 49–58. doi:10.1038/s41586-024-07146-0
- [33] Microsoft. 2022. Language Server Protocol Specification, Version 3.17. <https://microsoft.github.io/language-server-protocol/specifications/lsp/3.17/specification/>. Accessed: 2026-04-30.
- [34] Luc Moreau, Ben Clifford, Juliana Freire, Joe Futrelle, Yolanda Gil, Paul Groth, Natalia Kwasnikowska, Simon Miles, Paolo Missier, Jim Myers, Beth Plale, Yogesh Simmhan, Eric Stephan, and Jan Van den Bussche. 2011. The Open Provenance Model core specification (v1.1). *Future Generation Computer Systems* 27, 6 (2011), 743–756. doi:10.1016/j.future.2010.07.005

- [35] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. 2017. Ray: A Distributed Framework for Emerging AI Applications. *ArXiv abs/1712.05889* (2017). <https://api.semanticscholar.org/CorpusID:34552495> OSDI 2018.
- [36] Xiaoyi Nie and Haitao Zhang. 2025. Large Language Models Tool Retrieval and Context Compression via Dynamic Graph-Based Relation Modeling. *Academic Journal of Computing & Information Science* (2025). <https://api.semanticscholar.org/CorpusID:281120892>
- [37] OpenAI. 2023. GPT-4 Technical Report. *CoRR abs/2303.08774* (2023). arXiv:2303.08774 doi:10.48550/ARXIV.2303.08774
- [38] Greg Osuri and Adam Bozanich. 2020. AKT: Akash Network Token and Mining Economics. Akash Network Whitepaper. <https://akash.network/whitepapers/> Accessed: 2026-04-28.
- [39] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F. Christiano, Jan Leike, and Ryan Lowe. 2022. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, Sanmi Koyejo, S. Mohamed, A. Agarwal, Danielle Belgrave, K. Cho, and A. Oh (Eds.). http://papers.nips.cc/paper_files/paper/2022/hash/b1efde53be364a73914f58805a001731-Abstract-Conference.html
- [40] Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative Agents: Interactive Simulacra of Human Behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology* (San Francisco, CA, USA) (*UIST '23*). Association for Computing Machinery, New York, NY, USA, Article 2, 22 pages. doi:10.1145/3586183.3606763
- [41] Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Larivière, Alina Beygelzimer, Florence d'Alché Buc, Emily B. Fox, and H. Larochelle. 2020. Improving Reproducibility in Machine Learning Research (A Report from the NeurIPS 2019 Reproducibility Program). *J. Mach. Learn. Res.* 22 (2020), 164:1–164:20. <https://api.semanticscholar.org/CorpusID:214693121>
- [42] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. ChatDev: Communicative Agents for Software Development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 15174–15186. doi:10.18653/v1/2024.acl-long.810
- [43] Yujia Qin, Shengding Hu, Yankai Lin, Weize Chen, Ning Ding, Ganqu Cui, Zheni Zeng, Xuanhe Zhou, Yufei Huang, Chaojun Xiao, Chi Han, Yi Ren Fung, Yusheng Su, Huadong Wang, Cheng Qian, Runchu Tian, Kunlun Zhu, Shihao Liang, Xingyu Shen, Bokai Xu, Zhen Zhang, Yining Ye, Bowen Li, Ziwei Tang, Jing Yi, Yuzhang Zhu, Zhenning Dai, Lan Yan, Xin Cong, Yaxi Lu, Weilin Zhao, Yuxiang Huang, Junxi Yan, Xu Han, Xian Sun, Dahai Li, Jason Phang, Cheng Yang, Tongshuang Wu, Heng Ji, Guoliang Li, Zhiyuan Liu, and Maosong Sun. 2024. Tool Learning with Foundation Models. *ACM Comput. Surv.* 57, 4, Article 101 (Dec. 2024), 40 pages. doi:10.1145/3704435
- [44] Yujia Qin, Shi Liang, Yining Ye, Kunlun Zhu, Lan Yan, Ya-Ting Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. 2023. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. *ArXiv abs/2307.16789* (2023). <https://api.semanticscholar.org/CorpusID:260334759> ICLR 2024.
- [45] Frank Reinhold, Andreas Obersteiner, Stefan Hoch, Sarah Isabelle Hofer, and Kristina Reiss. 2020. The Interplay Between the Natural Number Bias and Fraction Magnitude Processing in Low-Achieving Students. *Frontiers in Education* 5 (2020). doi:10.3389/feduc.2020.00029
- [46] Alireza Rezaeadeh, Zichao Li, Ange Lou, Yuying Zhao, Wei Wei, and Yujia Bao. 2025. Collaborative Memory: Multi-User Memory Sharing in LLM Agents with Dynamic Access Control. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*. arXiv:2505.18279 [cs.AI] <https://arxiv.org/abs/2505.18279>
- [47] JV Roig. 2025. How Do LLMs Fail In Agentic Scenarios? A Qualitative Analysis of Success and Failure Scenarios of Various LLMs in Agentic Simulations. *ArXiv abs/2512.07497* (2025). <https://api.semanticscholar.org/CorpusID:283694152>
- [48] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. *ArXiv abs/2302.04761* (2023). <https://api.semanticscholar.org/CorpusID:256697342> NeurIPS 2023.
- [49] Renan Souza, Amal Gueroudji, Stephen DeWitt, Daniel Rosendo, Tirthankar Ghosal, Robert Ross, Prasanna Balaprakash, and Rafael Ferreira da Silva. 2025. PROV-AGENT: Unified Provenance for Tracking AI Agent Interactions in Agentic Workflows. *2025 IEEE International Conference on eScience (eScience)* (2025), 467–473. <https://api.semanticscholar.org/CorpusID:280526642>
- [50] Renan Souza, Timothy Poteet, Brian Etz, Daniel Rosendo, Amal Gueroudji, Woong Shin, Prasanna Balaprakash, and Rafael Ferreira da Silva. 2025. LLM Agents for Interactive Workflow Provenance: Reference Architecture and Evaluation Methodology. In *Proceedings of the SC '25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC Workshops '25)*. Association for Computing Machinery, New York, NY, USA, 2257–2268. doi:10.1145/3731599.3767582
- [51] Susan Leigh Star and Karen Ruhleder. 1996. Steps Toward an Ecology of Infrastructure: Design and Access for Large Information Spaces. *Information Systems Research* 7, 1 (1996), 111–134. doi:10.1287/isre.7.1.111
- [52] Maarten Van Steen and Andrew S. Tanenbaum. 2017. *Distributed Systems* (3rd ed.). distributed-systems.net. <https://www.distributed-systems.net/index.php/books/ds3/>
- [53] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Neural Information Processing Systems*. <https://api.semanticscholar.org/CorpusID:13756489>

- [54] Zixiang Wang, Mengjia Gong, Qiyu Sun, Jing Xu, Shuai Mao, Xin Jin, Qing-Long Han, and Yang Tang. 2026. Multi-Agent Systems: From Classical Paradigms to Large Foundation Model-Enabled Futures. *arXiv:2604.18133* [cs.AI] <https://arxiv.org/abs/2604.18133>
- [55] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, brian ichter, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh (Eds.), Vol. 35. Curran Associates, Inc., 24824–24837. https://proceedings.neurips.cc/paper_files/paper/2022/file/9d5609613524ecf4f15af0f7b31abca4-Paper-Conference.pdf
- [56] Jiaqi Wei, Yuejin Yang, Xiang Zhang, Yuhao Chen, Zhuang Xiang, Zhangyang Gao, Dongzhan Zhou, Guangshuai Wang, Zhiqiang Gao, Juntao Cao, Zijie Qiu, Xuming He, Qiang Zhang, Chenyu You, Shuangjia Zheng, Ning Ding, Wanli Ouyang, Nanqing Dong, Yu Cheng, Siqi Sun, Lei Bai, and Bowen Zhou. 2025. From AI for Science to Agentic Science: A Survey on Autonomous Scientific Discovery. *ArXiv abs/2508.14111* (2025). <https://api.semanticscholar.org/CorpusID:280691981>
- [57] Mark D. Wilkinson, Michel Dumontier, IJsbrand Jan Aalbersberg, Gabrielle Appleton, Myles Axton, Arie Baak, Niklas Blomberg, Jan-Willem Boiten, Luiz Olavo Bonino da Silva Santos, Philip E. Bourne, Jildau Bouwman, Anthony J. Brookes, Tim Clark, Mercè Crosas, Ingrid Dillo, Olivier Dumon, Scott C. Edmunds, Chris T. A. Evelo, Richard Finkers, Alejandra N. González-Beltrán, Alasdair J. G. Gray, Paul Groth, Carole A. Goble, Jeffrey S. Grethe, Jaap Heringa, Peter A. C. 't Hoen, Rob W. W. Hooft, Tobias Kuhn, Ruben G. Kok, Joost N. Kok, Scott J. Lusher, Maryann E. Martone, Albert Mons, Abel Laerte Packer, Bengt Persson, Philippe Rocca-Serra, Marco Roos, Rene C. van Schaik, Susanna-Assunta Sansone, Erik Anthony Schultes, Thierry Sengstag, Ted Slater, George O. Strawn, Morris A. Swertz, Mark Thompson, Johan van der Lei, Erik M. van Mulligen, Jan Velterop, Andra Waagmeester, Peter Wittenburg, Katy Wolstencroft, Jun Zhao, and Barend Mons. 2016. The FAIR Guiding Principles for scientific data management and stewardship. *Scientific Data* 3 (2016). doi:10.1038/sdata.2016.18
- [58] Michael J. Wooldridge. 2009. *An Introduction to MultiAgent Systems, Second Edition*. Wiley.
- [59] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryan W. White, Doug Burger, and Chi Wang. 2023. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. <https://api.semanticscholar.org/CorpusID:263611068> COLM 2024.
- [60] Stefan Wuchty, Benjamin F. Jones, and Brian Uzzi. 2007. The Increasing Dominance of Teams in Production of Knowledge. *Science* 316, 5827 (2007), 1036–1039. doi:10.1126/science.1136099
- [61] Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob Foerster, Jeff Clune, and David Ha. 2025. The AI Scientist-v2: Workshop-Level Automated Scientific Discovery via Agentic Tree Search. *ArXiv abs/2504.08066* (2025). <https://api.semanticscholar.org/CorpusID:277741107>
- [62] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. ReAct: Synergizing Reasoning and Acting in Language Models. *ArXiv abs/2210.03629* (2022). <https://api.semanticscholar.org/CorpusID:252762395> ICLR 2023.
- [63] Mario Zechner. 2026. *pi-mono: AI Agent Toolkit – Coding Agent CLI, Unified LLM API, TUI & Web UI Libraries, Slack Bot, vLLM Pods*. <https://github.com/badlogic/pi-mono> Accessed: 2026-04-25.
- [64] Zeyu Zhang, Quanyu Dai, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. 2025. A Survey on the Memory Mechanism of Large Language Model-based Agents. *ACM Trans. Inf. Syst.* 43, 6, Article 155 (Sept. 2025), 47 pages. doi:10.1145/3748302
- [65] Filip Zoubek, Tomasz Miksa, and Andreas Rauber. 2025. Conceptual service architecture to synchronise research data management services using machine-actionable data management plans. *ACM Trans. Manage. Inf. Syst.* 16, 3, Article 23 (May 2025), 23 pages. doi:10.1145/3712014